DeLonghi PrimaDonna ELITE ��� BLE MITM

DeLonghi PrimaDonna ELITE — BLE MITM міст + Home Assistant + WLED-матриця

Останнє оновлення: серпень 2026, прошивка v4.15-C6-MITMworkflow

🔄 Як користуватись цим документом у новому чаті

Встав весь цей файл на початку нового чату разом із фразою: “Продовжуємо проєкт DeLonghi BLE MITM, ось повна документація поточного стану”. Claude одразу зрозуміє весь контекст: протокол, що реалізовано, як компілювати, які автоматизації потрібні.

Якщо потрібен сам код прошивки — прикріпи файл `delonghi_c6_mitm_v415_workflow.ino` (чи новішу версію, якщо вже змінювали) до повідомлення.

1️⃣ Опис роботи приладу

Що це

ESP32-C6 виступає як MITM-міст (man-in-the-middle) між: - Кавомашиною DeLonghi PrimaDonna ELITE ECAM 656.75 (MAC 00:1E:C0:30:C0:45, серійник 219006CP16031740064) - Офіційним застосунком CoffeeLink на телефоні

ESP32-C6 одночасно: - central-роль → підключається до реальної машини по BLE - peripheral-роль → рекламує себе телефону як клоновану машину (той самий MAC, той самий сервіс)

Завдяки цьому міст бачить усі команди й відповіді між застосунком і машиною, і водночас може сам ін’єктувати власні команди (наприклад, від Home Assistant), не заважаючи офіційному застосунку працювати паралельно.

Протокол (повністю розшифрований)

Кожна команда починається з `0d` (старт), далі довжина, опкод, дані, і чек-сума = XOR усіх попередніх байтів XOR 0x55

Родина опкодів 82 f0 — приготування напою:

```
0d 0b 82 f0 <recipeID> <action> <вода_старший> <вода_молодший> <молоко_старший> <молоко_молодший>
<міцність> <checksum>
```

action: 0x00=зберегти, 0x01=старт, 0x04=стоп Вода/кава: мл×2 (16-біт). Молоко: сек×10 (16-біт). Міцність: (рівень×16) XOR mode, де mode=1 для SAVE, =2 для СТАРТ/СТОП recipeId: еспресо=1, кава=2, капучино=7, латте макіато=8, гаряча вода=16

A9 F0 — перемкнути активний профіль на машині: `0d 05 A9 F0 <номер_профілю_1-6> <checksum>`

A6 F0 — прочитати поточні параметри напою з активного профілю: `0d 07 A6 F0 <номер_профілю> <recipeId> <recipeId> <checksum>` → відповідь 11 байт: `d0 0a A6 F0 <профіль> <вода_ст> <вода_мол> <молоко_ст> <молоко_мол> <міцність_комбо> <checksum>`

0x75 (26 байт) — основний статус-пакет: byte[25]=% приготування, byte[19]/[20]=внутрішній стан

машини (потрібен для дозволу SAVE — див. нижче)

0x70 — MonitorV1 (ця машина, схоже, ніколи не шле цей опкод — усе через 0x75)

SAVE (action=0x00) реально запишеться в постійну пам'ять машини ЛИШЕ якщо `byte[19]==7 && byte[20]==0` в останньому статус-пакеті (те саме, що офіційний застосунок перевіряє перед збереженням)

Прошивка вже вміє (детальніше в розділі 3)

- Показувати кавові напої, варити з обраними параметрами
- Зберігати "свої" параметри локально (в ESP32) БЕЗ впливу на машину
- Записувати параметри напряму в один із 6 профілів машини
- Читати поточні параметри з будь-якого профілю (з візуальним порівнянням "було/стало")
- Обирати джерело параметрів для приготування "на льоту": власна пам'ять ESP32, чи один із 6 профілів машини

2️⃣ Опис роботи світлодіодного індикатора

RGB-світлодіод на самому ESP32 (стан з'єднання)

- Червоний, стабільний — щойно увімкнувся, ще не підключився нізвідки
- Фіолетовий — підключається до Wi-Fi
- Колір визначається станом BLE до машини (базовий колір overlay)
- Коротке біле блимання кожні ~4с — Wi-Fi OK
- Жовте блимання — Wi-Fi недоступний (наприклад, роутер перезавантажився)

WLED-матриця 16×16 (окремий пристрій, керується через Home Assistant)

Показує два типи інформації по черзі, залежно від того, що зараз важливіше:

Коли машина активна (прогрівається/готує/щойно закінчила):

Preset	Що показує
1	Прогрів (анімація "Black Hole")
2-12	Чашка кави, заповнення 0-100% (по 10%)
13	"Готовність" — пульсуюча веселкова чашка
14	Велика анімована крапля води (немає води)
15	Велика анімована іконка смітника (гуща повна)
16	Маленька іконка кавового зерна з написом "КАВА" (немає зерен)

Коли машина спить — показує погоду/годинник/дату циклічно (щохвилини перемикається: час → дата → температура → вологість → тиск), з автоматичною яскравістю від датчика освітлення.

Вимикається повністю після 3 годин темряви (якщо кухня вимкнена), і знову вмикається на світанку чи коли кухню вмикають вручну вночі.

Повна логіка (коли показувати погоду проти "вимкнено") — див. розділ 5, автоматизація "Погода — повне керування".

> ⚠ **Важливий нюанс (взаємодія кавових автоматизацій із нічним режимом):** "Прогрів на матриці" та "Вимкнути матрицю після прогріву" реагують на переходи стану машини (waking / ready / asleep) **незалежно від того, темно зараз чи ні** — вони не перевіряють нічний режим взагалі. Тобто якщо матриця щойно вимкнулась через "темно + 3 години + кухня вимкнена", а потім хтось вмикає кавомашину, матриця коротко "оживе" й покаже прогрів/чашку/"Готовність" — і це станеться ДВІЧІ: коли машина стає `ready`, і знову коли вона засинає (`asleep`). Це не залишає матрицю увімкненою назавжди: автоматизація "Погода — повне керування" має власний `time_pattern`-тригер раз на хвилину (`minutes: "/1"`), який щоразу перераховує "чи має бути видно"

> з нуля — тож якщо умова “темно + 3h + кухня off” усе ще виконується, матриця сама вимкнеться максимум протягом ~1 хвилини після того, як кавові автоматизації її розбудили. Підсумок: коротке “блимання” (до хвилини) — очікувана поведінка, застрягання назавжди — ні.

3️⃣ Що реалізовано в прошивці (станом на v4.15)

Базове керування

- Приготування 5 напоїв (кава/еспресо/капучино/латте макіато/гаряча вода) + стоп-кнопки
- Веб-дашборд на ESP32 (головна сторінка) з живим статусом через WebSocket
- MQTT-інтеграція для Home Assistant (стан машини, кнопки напоїв, зображення, прогрес приготування)
- Wi-Fi watchdog (автоперепідключення), OTA-оновлення прошивки через веб
- `/info` сторінка — QR-код, лічильник напоїв за день, чек-лист сервісних тривог, діагностика (останній надісланий код, останній отриманий пакет)

Налаштування напоїв (панель кожного напою)

Повзунки: вода (мл), міцність (1-5), молоко (сек, де є)

- 🖫 **Зберегти** — записує ЛИШЕ в пам’ять ESP32 (NVS), машину не чіпає
- ↺ **Дефолт** — скидає до заводських перевірених значень
- 📌 **В профіль** — циклічно обирає профіль машини (1→2→…→6→1), при кожному виборі:
  - Перемикає активний профіль на машині (A9F0)
  - Читає поточні параметри цього профілю (A6F0)
  - Малює червоні крапки на повзунках у місці поточних (профільних) значень — рухаєш повзунки, крапки лишаються, видно “було”/“стане”
  - Коли “В профіль” активний, кнопка “Зберегти” сама стає “Зберегти в профіль” і записує поточні (можливо, змінені) значення повзунків саме в обраний профіль машини (з перевіркою готовності машини — до 8с очікування)

Джерело параметрів для приготування (кнопка “Увімкнути” на головній)

- Поки машина спить — звичайна кнопка “Увімкнути” (будить машину)
- Щойно машина стає готова — кнопка сама (без кліку) перетворюється на індикатор джерела: 📶 ESP32 (синій)
- Клік циклічно перемикає: ESP32 → Профіль 1 → 2 → … → 6 → знову ESP32, з унікальним кольором для кожного
- При виборі профілю (не ESP32) — одразу шле A9F0 (перемкнути) + пробний A6F0 для кави (лише перевірка зв’язку, значення не використовуються)
- Коли натискаєш будь-який напій:
  - Якщо джерело “ESP32” — варить зі збережених у пам’яті ESP32 значень (як завжди)
  - Якщо джерело “Профіль N” — спочатку читає параметри САМЕ цього напою з профілю N (A6F0), і варить цими значеннями; якщо прочитати не вдалось — тихо повертається до значень ESP32
- Коли машина знову засинає — кнопка сама скидається на “Увімкнути”

Не реалізовано / відкриті питання

Візуальний “тест на зв’язок” при виборі профілю (пункт вище) поки не показує окремого статусу успіху/невдачі користувачу — лише блимає під час читання

4️⃣ Автоматизації, потрібні в Home Assistant

Автоматизація	Призначення

“Погода — повне керування”	Головна логіка WLED-матриці вночі: показ погоди/вимкнення/яскравість/реакція на кухню — усе в одній автоматизації (деталі в розділі 5)
“Примусова погода вимкнена — відновити режим”	Коли вимикаєш ручний форс погоди, відновлює правильний показ (прогрів/готовність)
“Прогрів на матриці”	Показує анімацію прогріву на матриці
“Вимкнути матрицю після прогріву”	Показує “Готовність” коли машина стає ready/asleep
“Кавова чашка на матриці”	Показує % приготування під час варіння (з захистом від показу поверх активних попереджень)
“Попередження на матриці”	Показує іконки вода/гуща/зерна коли є проблема

Плюс два прості скрипти: `script.uvimknuti_matritsiu` (примусово увімкнути матрицю) і `script.vimknuti_matritsiu` (примусово вимкнути, з “запам’ятовуванням” ручного вимкнення).

Повний YAML-текст усіх автоматизацій, помічників (helpers) і `rest_command` — окремий файл, див. нижче.

5️⃣ Файл з автоматизаціями та помічниками

Дивись прикріплений файл `HA_автоматизації_і_помічники.yaml` — там повний, готовий до вставки текст усього необхідного.

Налаштування Arduino IDE (для ручної компіляції на своєму комп’ютері)

Крок 1 — встановити Arduino IDE

Завантаж і встанови Arduino IDE 2.x (безкоштовно, Windows/Mac/Linux).

Крок 2 — додати підтримку плат ESP32

1. File → Preferences (чи Arduino IDE → Settings на Mac)
2. У полі “Additional Boards Manager URLs” встав: `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`
3. OK
4. Tools → Board → Boards Manager, знайди “esp32” (автор: Espressif Systems)
5. Встанови версію саме 3.2.1 (виберіть її з випадного списку версій перед натисканням “Install” — новіші/старіші версії можуть не збігтись з бібліотеками нижче)

Крок 3 — обрати плату й налаштування

1. Tools → Board → esp32 → ESP32C6 Dev Module
2. Tools → Partition Scheme → Minimal SPIFFS (…)
3. Tools → CDC On Boot → Enabled
4. Порт — обери свій, коли підключиш ESP32-C6 по USB

Крок 4 — встановити бібліотеки

Sketch → Include Library → Manage Libraries…, знайди й встанови кожну (натисни на назву, обери версію з випадного списку версій ПЕРЕД “Install”):

Бібліотека	Автор	Версія	Призначення
NimBLEArduino	h2zero	2.3.4	Легка BLE-бібліотека (центральна + периферійна роль одночасно)

PubSubClient	Nick O'Leary (knolleary)	остання стабільна (2.8+)	MQTT-клієнт для інтеграції з Home Assistant
WiFiManager	tzapu	остання стабільна (2.0.17+)	Портал налаштування Wi-Fi через веб-інтерфейс
Adafruit NeoPixel	Adafruit	остання стабільна (1.12+)	Керування RGB-світлодіодом на платі
WebSockets	Markus Sattler (Links2004)	остання стабільна (2.4+)	WebSocket-сервер для живого дашборду

⚠ NimBLE-Arduino обов'язково версії 2.3.4 — новіші (3.0.0-dev тощо) мають несумісні зміни API, прошивка не скомпілюється без правок коду.

Решта бібліотек (WiFi, WebServer, Preferences, HTTPClient тощо) — вбудовані в ESP32-ядро, окремо встановлювати не треба.

Крок 5 — відкрити й скомпілювати

1. Файл `.ino` обов'язково повинен лежати у папці з такою самою назвою (наприклад, `delonghi_c6_mitm_v30/delonghi_c6_mitm_v30.ino`) — Arduino IDE вимагає цього
2. Відкрий `.ino`-файл через File → Open
3. Sketch → Verify/Compile (перевірити компіляцію) чи Sketch → Upload (залити напряму на підключений ESP32)
4. Готовий `.bin` для OTA-оновлення (без USB) можна знайти після компіляції в Sketch → Export Compiled Binary — файл з'явиться в тій самій папці, що й `.ino`

🤖 Інструкція: як скомпілювати .bin у новому чаті

Якщо продовжуєш у новому чаті Claude (з увімкненим кодовим виконанням/computer use):

1. Встав повідомлення: "Ось код прошивки DeLonghi MITM, потрібно внести зміни й скомпілювати новий .bin" + прикріпи `.ino`-файл
2. Опиши, що саме змінити
3. Claude сам налаштує середовище компіляції — ось точні кроки, які він має виконати (для довідки, якщо щось піде не так):

```
        # 1. Завантажити arduino-cli
        curl -fsSL -o arduino-cli.tar.gz "https://github.com/arduino/arduino-
cli/releases/download/v1.1.1/arduino-cli_1.1.1_Linux_64bit.tar.gz"
        tar xzf arduino-cli.tar.gz
        export HOME=/home/claude
        export PATH=/home/claude:$PATH

        # 2. Налаштувати індекс плат ESP32
        arduino-cli config init --additional-urls "https://raw.githubusercontent.com/espressif/arduino-esp3
pages/package_esp32_index.json" --overwrite
        arduino-cli core update-index

        # 3. downloads.arduino.cc заблокований у цьому середовищі — потрібен "фейковий"
        # package_index.json, що вказує на локальний http.server (порт 8899) замість
        # заблокованого dfu-util/serial-discovery/mdns-discovery.
        # (Claude знає, як зробити ці фейкові .tar.gz-архіви правильної структури —
        # якщо забув, це найскладніша частина, варто дати йому час розібратись)

        # 4. Встановити ESP32 core 3.2.1
        arduino-cli core install esp32:esp32@3.2.1

        # 5. Бібліотеки (git clone --depth 1 у ~/Arduino/libraries/):
        # - NimBLE-Arduino, автор h2zero, ГІЛКА САМЕ 2.3.4 (не master/HEAD!)
        # git clone --depth 1 --branch 2.3.4 https://github.com/h2zero/NimBLE-Arduino.git
        # - PubSubClient, автор Nick O'Leary (knolleary)
```

```
        # https://github.com/knolleary/pubsubclient.git
        # - WiFiManager, автор tzapu
        # https://github.com/tzapu/WiFiManager.git
        # - Adafruit NeoPixel, автор Adafruit
        # https://github.com/adafruit/Adafruit_NeoPixel.git
        # - WebSockets, автор Markus Sattler (Links2004)
        # https://github.com/Links2004/arduinoWebSockets.git (папка має називатись "WebSockets")

        # 6. ctags (теж окремо блокований):
        FILE=$(curl -sL https://github.com/arduino/ctags/releases/expanded_assets/5.8-arduino11 | grep -oE
'ctags-5\.8-arduino11-x86_64-pc-linux-gnu\.tar\.bz2' | head -1)
        mkdir -p ~/.arduino15/packages/builtin/tools/ctags/5.8-arduino11
        cd ~/.arduino15/packages/builtin/tools/ctags/5.8-arduino11
        curl -fsSL -o ctags.tar.bz2 "https://github.com/arduino/ctags/releases/download/5.8-arduino11/$FILE
        tar xjf ctags.tar.bz2 && chmod +x ctags

        # 7. Компіляція
        mkdir -p sketch_build/delonghi_c6_mitm_v30
        cp delonghi_c6_mitm_v30.ino sketch_build/delonghi_c6_mitm_v30/
        arduino-cli compile --fqbn "esp32:esp32:esp32c6:PartitionScheme=min_spiffs,CDCOnBoot=cdc"
sketch_build/delonghi_c6_mitm_v30 --export-binaries
```

Важливо: файл `.ino` обов'язково повинен називатись так само, як папка, в якій лежить (тут — delonghi_c6_mitm_v30), інакше Arduino не скомпілює.

⚠ USB CDC вимкнено з v4.42-cdcoff — важливо знати

З версії v4.42 і далі прошивка компілюється з `CDCOnBoot=default` (вимкнено), а не `CDCOnBoot=cdc`, яким компілювались усі попередні версії.

Чому: з `CDCOnBoot=cdc` і без фізично підключеного USB-хоста (стаціонарна установка, як зараз) внутрішній буфер `Serial.print()`/`println()`/`printf()` рано чи пізно заповнюється (ніхто його не вичитує), і **всі подальші виклики зависають назавжди**, чекаючи місця в буфері. Це підтверджено наживо: під час реального варіння (де частота подій різко зростає) весь `loop()` фактично зупинявся — і веб-дашборд, і сама обробка BLE.

Що це означає практично: - USB Serial Monitor **більше не працює** для живої діагностики (`Serial.begin()` тепер веде в нікуди — CDC-периферія не ініціалізується при старті). - Уся діагностика тепер іде через веб: **`/debug_log`** (кільцевий буфер останніх ~100 подій — `func`/`exec`, `brew_progress`, збірка BLE-пакетів) — не залежить від USB, працює завжди. - На всякий випадок у коді (з v4.42) також додано `if (Serial)`-захист на всі 48 місць виклику `Serial.*` — тобто навіть якби CDC колись знову увімкнули, блокування вже не мало б повторитись.

Якщо колись знадобиться саме "сира", живцем USB-діагностика (глибша за те, що вже пише `/debug_log`): 1. Візьми прошивку **v4.15** (перша версія цієї серії, ще до всіх self-poll/детекція-варіння фіксів, скомпільована з `CDCOnBoot=cdc`) — чи будь-яку іншу версію ДО v4.42, яка теж мала CDC увімкненим. 2. Підключи USB, відкрий Serial Monitor, залодий потрібний сценарій. 3. Після — поверни поточну стабільну версію (на момент цього запису — **v4.42-C6-MITM-cdcoff**) назад.

Компіляція: якщо колись треба зібрати версію з увімкненим CDC знову — команда компіляції відрізняється лише одним прапорцем:

```
        # з CDC (USB Serial Monitor працює, ризик зависання без хоста):
        arduino-cli compile --fqbn "esp32:esp32:esp32c6:PartitionScheme=min_spiffs,CDCOnBoot=cdc" ...

        # без CDC (стабільно стаціонарно, лише /debug_log):
        arduino-cli compile --fqbn "esp32:esp32:esp32c6:PartitionScheme=min_spiffs,CDCOnBoot=default" ...
```

💡 Ключові нюанси й уроки з двотижневого шляху

1. **Arduino String-конкатенація повільна й фрагментує пам’ять** — при кожному додаванні (`+=`) відбувається переалокація. Якщо рядок будується у циклі, що часто виконується (наприклад, при кожному BLE-пакеті) — використовуй фіксований char-буфер замість String. Для великих HTML-сторінок — заздалегідь `html.reserve(N)`.
2. **delay() всередині HTTP-обробника блокує loop()** — якщо в loop() є логіка обробки BLE-фрагментів (як у цьому проєкті), і HTTP-обробник чекає на відповідь через delay()-цикл, він сам собі “заважає” отримати цю відповідь. Рішення: викликати логіку обробки фрагментів САМОМУ всередині циклу очікування, а не покладатись на loop().
3. **НА автоматизації, що незалежно реагують на ту саму подію, рано чи пізно “зіштовхуються”** — єдиний надійний підхід: ОДНА автоматизація на кожну логічну задачу, яка щоразу рахує потрібний стан “з нуля”, без прапорців-станів, що можуть розсинхронізуватись.
4. **now() у Jinja2-шаблонах HA — timezone-aware**, а `as_datetime` на рядку з `input_datetime` — HI, тому пряме віднімання падає з помилкою. Завжди `as_timestamp()` обидві сторони перед відніманням.
5. **Byte-офсети всередині одного й того самого опкоду можуть різнитись залежно від версії пакета** — MonitorData v1 (0x70) і v2 (0x75) мають різні позиції для тих самих логічних полів. Завжди перевіряй, яку САМЕ версію реально шле твоя машина.
6. **Подієві автоматизації (реагують на конкретний перехід стану) можуть тимчасово “перебити” стан, розрахований іншою, періодичною автоматизацією** — якщо є “нічний”/фоновий режим з власним `time_pattern`-тіком, короткочасне “перебивання” від подієвої автоматизації (наприклад, кавомашина будить матрицю) саморозсмоктується на наступному тіку, а не залишається назавжди. Перевіряй, чи є такий тік, перш ніж вважати подібну поведінку багом.